BITS Pilani WILP | AIMLCZG523 Machine Learning Operations | Assignment 01

Heart Disease Risk Prediction: an End-to-End MLOps Pipeline

From UCI data to a monitored, containerised API on Kubernetes

Student	Repository	Submission
Prince Sharma | BITS ID: 2025AE05605	https://github.com/princevashist1988/ML-Ops-Heart-Disease	20 October 2026

Summary

A binary classifier predicts the presence of heart disease (more than 50% narrowing of a major vessel) from 13 routine clinical attributes in the UCI Cleveland dataset, and is delivered as a production-style service. The pipeline covers checksum-verified data acquisition, exploratory analysis, a single leakage-free scikit-learn pipeline, a tracked hyper-parameter search over three model families (88 candidates, all logged to MLflow), a registered and packaged champion model, a quality-gated GitHub Actions workflow, a hardened Docker image, Kubernetes manifests plus a Helm chart, and Prometheus/Grafana monitoring with request logging and an input-drift signal. Every stage was executed end to end; the screenshots in this report are from those runs.

Result	Value
Champion model	Logistic regression (L2, C = 0.1, balanced class weights)
Cross-validated ROC-AUC (5-fold x 3 repeats)	0.899 +/- 0.039 (random forest 0.894, gradient boosting 0.877)
Hold-out, 61 patients, threshold 0.31	ROC-AUC 0.961 | recall 1.00 | precision 0.72 | accuracy 0.82
External validation, Hungarian site (293)	ROC-AUC 0.894 | recall 0.80 | precision 0.75
Engineering quality	57 unit tests, 91% coverage, lint-clean; GitHub Actions 5/5 jobs green; reproducible from a clean checkout
Serving (Kubernetes, live traffic)	p50 ~15 ms, p95 ~40 ms; rolling update with 500 requests, 0 failed

Key design decisions

Decision	Choice	Reason
Missing values	Imputed inside the pipeline, fitted on training folds only	No leakage; identical logic at inference; API accepts missing ca/thal
Model ranking	ROC-AUC under repeated stratified 5-fold CV (x3)	Threshold-free; one split of 242 rows varies by several AUC points
Hold-out set	Used for reporting only, never for selection	Selecting on the test set inflates the reported estimate
Decision threshold	Max F2 on out-of-fold training predictions (0.31)	Screening: a missed case costs more than a follow-up test
Engineered features	Toggled as a grid parameter (ablation)	Kept only if CV improves - it did not for the champion
Data integrity	Fallback sources accepted only with matching SHA-256	Training data can never change silently
Model files	skops with a type allow-list; joblib for the API	Loading a tampered artefact cannot execute code
Serving scale-out	One uvicorn worker per pod; replicas + HPA	Exact per-process metrics; Kubernetes-native scaling
Logging and drift	JSON request logs without clinical inputs; live vs training means	Traceability and early drift warning without storing patient data

1. Architecture

Five stages share one repository and one model contract. Data are acquired and validated once; a single scikit-learn pipeline is tuned, tracked and packaged; GitHub Actions gates every change; the packaged model is served by a FastAPI container on Kubernetes; Prometheus, Grafana and structured logs observe it. Solid arrows are data/control flow, dashed arrows fallback or optional paths.

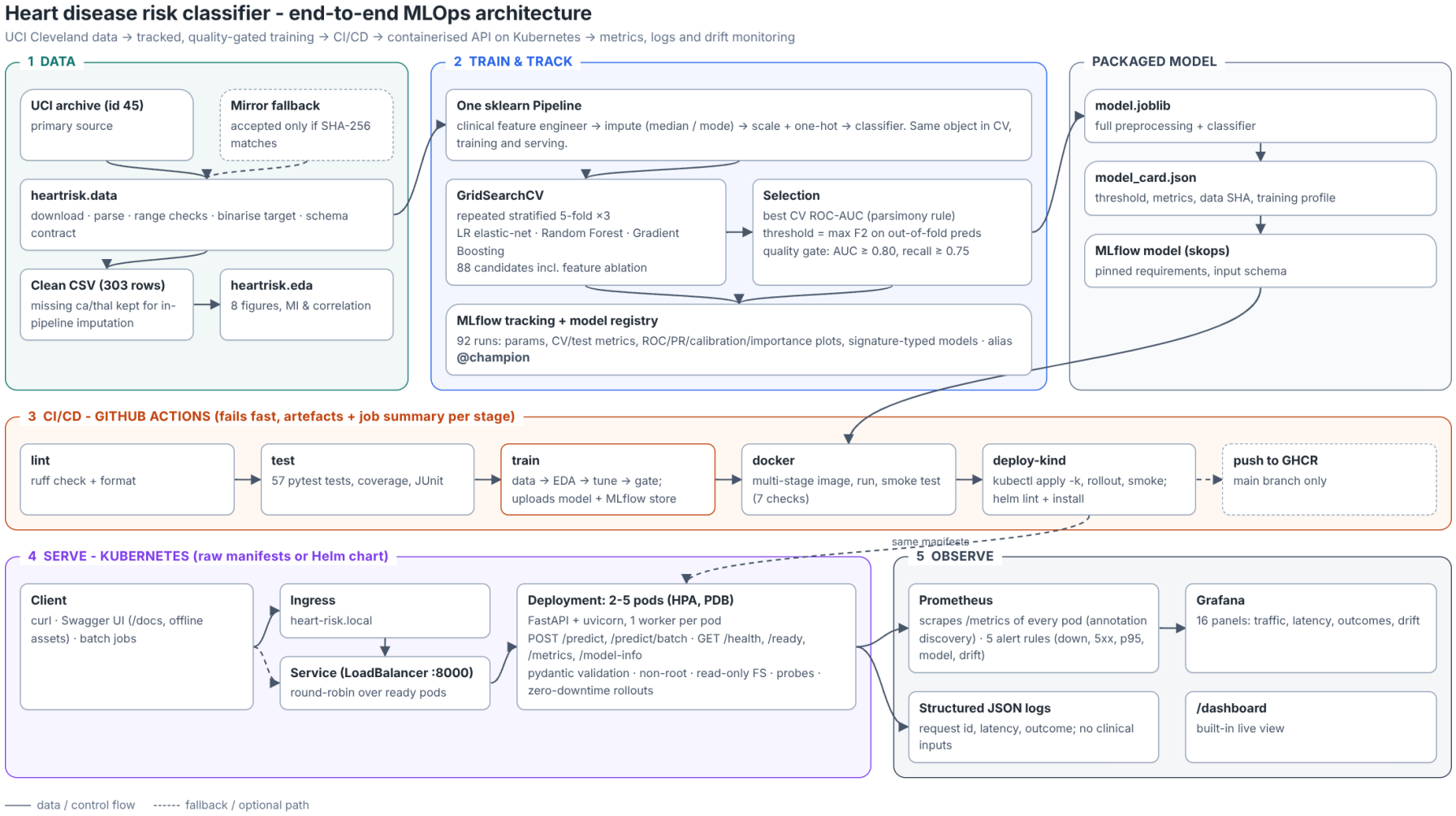


Figure 1. End-to-end architecture.

2. Setup and installation

Requirements: Python 3.11-3.13, Docker, and for deployment a local Kubernetes (Minikube, Docker Desktop or kind). All dependencies are pinned in `requirements.txt` (full environment, which also installs the project as an editable package), `requirements-serve.txt` (minimal runtime used in the container) and `environment.yml` (Conda).

```
git clone https://github.com/princevashist1988/ML-Ops-Heart-Disease.git heart-risk-mlops && cd
heart-risk-mlops
python3.12 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
make data && make eda && make train && make test      # or: python -m heartrisk.<stage>
docker build -t heart-risk-api:1.0.0 . && docker run -p 8000:8000 heart-risk-api:1.0.0
kubectl apply -k deploy/k8s && kubectl apply -k deploy/monitoring   # or: helm install ...
deploy/helm/heart-risk-api
scripts/ci_local.sh        # the CI stages locally: lint -> test -> data -> train -> docker ->
smoke test
```

Reproducibility was verified by copying the repository without any generated state (no virtual environment, raw data, MLflow store or logs), installing from `requirements.txt` and running `scripts/ci_local.sh`: all six stages passed and the training run reproduced the original cross-validation scores to four decimals (Figure 11).

3. Data acquisition and exploratory analysis

3.1 Acquisition and cleaning

- `python -m heartrisk.data download` tries the UCI archive (zip, then legacy file URL). If unreachable it falls back to public copies of `processed.cleveland.data`, accepted **only if the SHA-256 matches a pinned hash** (three independent copies agree on it). A Weka ARFF copy was rejected for training after a row-by-row comparison showed it drops one record and duplicates another. Every download writes `data/raw/provenance.json`.
- Cleaning: `?` markers become missing values (4 in `ca`, 2 in `thal`); physiologically impossible values become missing (none in Cleveland, e.g. cholesterol = 0 in the Hungarian file); the 0-4 severity label is binarised (`num > 0`); exact duplicates are dropped (none). Missing values are deliberately kept and imputed inside the model pipeline, so imputation is fitted on training folds only and the same logic runs at inference.
- A schema contract (`data.validate`: columns, ranges, category domains, binary non-constant target) runs before training, in the tests and in CI. The cleaned 303 x 14 CSV is committed; the Hungarian file (294 patients) is kept as an external validation set.

3.2 Findings

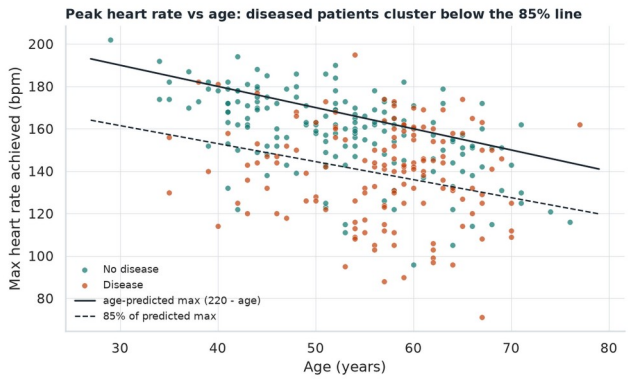


Figure 2. Peak heart rate vs age with the age-predicted maximum.

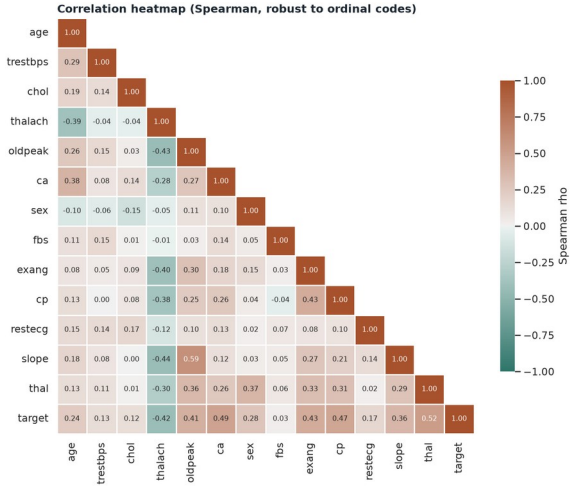


Figure 3. Spearman correlation heat-map.

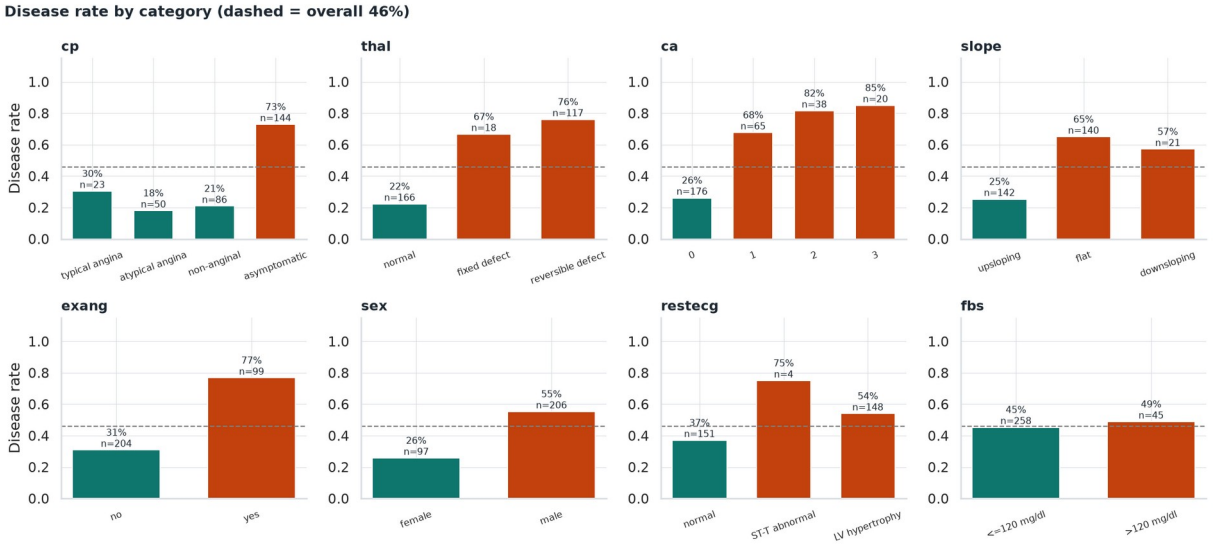


Figure 4. Disease rate by category (dashed line = overall 46%); n per bar.

- **Near-balanced target** (164 healthy vs 139 diseased, 54/46): accuracy is meaningful, but ROC-AUC ranks models because it is threshold-free, and recall is reported because a missed diagnosis is the costly error.
- **Strongest signals are categorical**: `thal` (rho = 0.52), `ca` (0.49), `cp` (0.47), `exang` (0.43); `thalach` is the strongest numeric signal (-0.42). Asymptomatic chest pain carries 73% prevalence versus 18-30% for anginal types, so `cp` is one-hot encoded rather than treated as ordinal.
- **Age-adjusted heart rate** (Figure 2): patients reaching less than 85% of the age-predicted maximum (220 - age) have 72% prevalence versus 35% above it. This motivated the engineered feature `hr_pct_predicted`, tested as an ablation.
- **Outliers** (9 in resting BP, 5 in cholesterol, 5 in ST depression by the IQR rule) are plausible clinical extremes and were kept; scaling and regularisation handle them. Fasting blood sugar shows almost no association (rho = 0.03).
- **External site shift**: the Hungarian file is missing `ca` for 99% and `thal` for 90% of patients - a realistic deployment shift used to stress-test the model.

4. Feature engineering and model development

All transformations live in one scikit-learn `Pipeline`, so cross-validation, final training and the API execute identical code: `ClinicalFeatureEngineer` (adds `hr_pct_predicted` and a stage-2 hypertension flag, toggled by a hyper-parameter) -> cast to float -> `ColumnTransformer` (numeric: median imputation + standardisation; binary: mode imputation; categorical: mode imputation + one-hot with unknown categories ignored) -> classifier.

The data are split once into a stratified 80/20 train/hold-out partition (242/61). Each model family is tuned with `GridSearchCV` using **repeated stratified 5-fold CV (3 repeats)**, scored on ROC-AUC, accuracy, precision, recall and F1. Repeated CV was chosen because with 242 training rows a single 5-fold split varies by several AUC points. The hold-out set is never used for selection.

Model family	Search space (all with engineered features on/off)	Best configuration	CV ROC-AUC	Train-CV gap
Logistic regression	C in {0.03, 0.1, 0.3, 1}; l1_ratio in {0, 0.5, 1}; class weight (48 fits)	C = 0.1, L2, balanced, no engineered features	0.899 +/- 0.039	0.026
Random forest	300 trees; depth {None, 4, 8}; min leaf {1, 4}; max features {sqrt, 0.5} (24)	unlimited depth, leaf 4, sqrt	0.894 +/- 0.041	0.082
Gradient boosting	trees {100, 250}; learning rate {0.03, 0.1}; depth {2, 3}; subsample 0.8 (16)	100 trees, 0.03, depth 2, engineered on	0.877 +/- 0.044	0.089

Selection. The champion is the family with the best mean CV ROC-AUC, with a configured parsimony rule (prefer the interpretable model if within 0.01). Logistic regression won outright and also has the smallest train-CV gap, i.e. the least over-fitting. Its coefficients are clinically coherent: vessels coloured (`ca`), asymptomatic chest pain, male sex and reversible thallium defect raise risk; a normal thallium scan and higher peak heart rate lower it.

Ablation. Because `features__enabled` is a grid parameter, the search measured the engineered features directly: best CV AUC changed by -0.002 (LR), -0.003 (RF) and +0.001 (GB), well inside one standard deviation. Despite the

strong univariate signal in Figure 2, the linear model already combines `age` and `thalach`, so the simpler feature set was kept. The transformer stays in the pipeline (disabled) so the ablation is reproducible.

Operating threshold. For a screening use case the decision threshold is chosen to maximise F2 (recall weighted four times precision) on **out-of-fold predictions of the training set**, subject to precision >= 0.6. The resulting threshold, 0.31, is stored in the model card and served by the API.

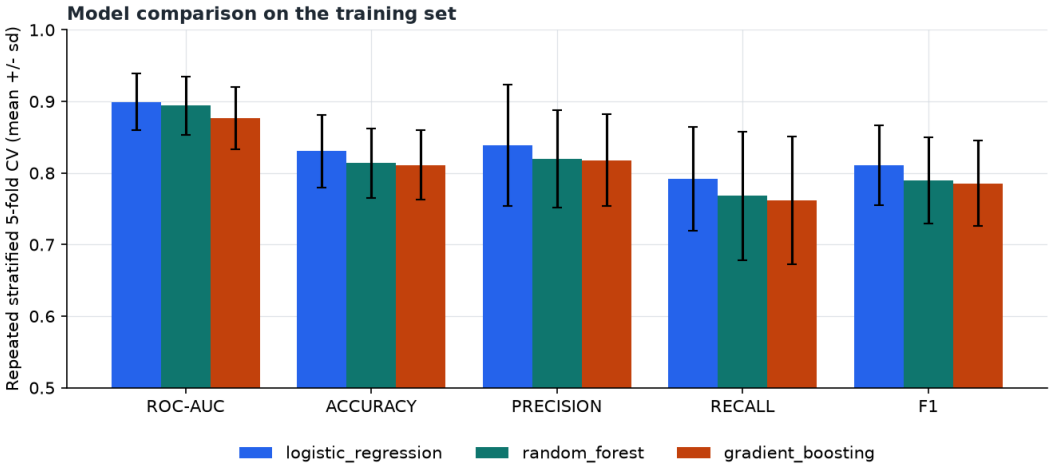


Figure 5. Cross-validated metrics (mean +/- sd) for the best configuration of each family.

Hold-out metric (n = 61)	Threshold 0.50	Threshold 0.31 (deployed)	External site, threshold 0.31 (n = 293)
ROC-AUC / PR-AUC	0.961 / 0.949	0.961 / 0.949	0.894 / 0.857
Recall (sensitivity)	0.929	1.000	0.802
Precision	0.839	0.718	0.752
Specificity	0.848	0.667	0.850
Accuracy / F1	0.885 / 0.881	0.820 / 0.836	0.833 / 0.776
Missed cases (FN) / false alarms (FP)	2 / 5	0 / 11	21 / 28
Brier score	0.091	0.091	0.133

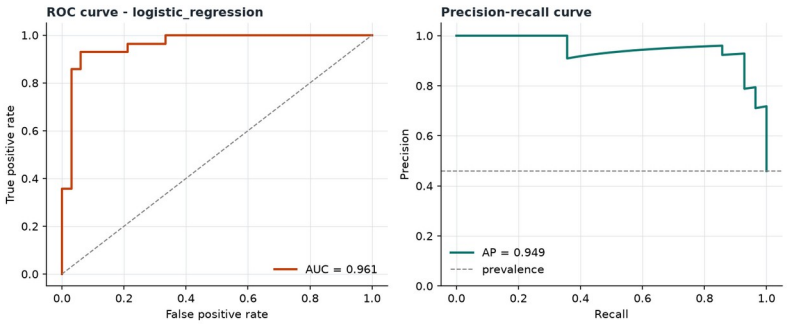


Figure 6. Hold-out ROC and precision-recall curves.

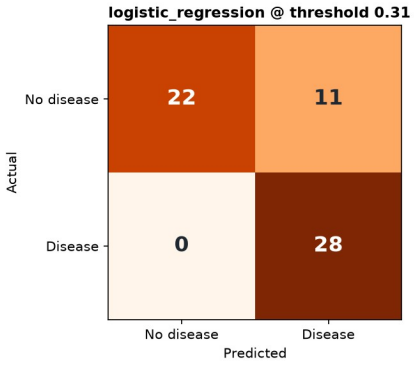


Figure 7. Hold-out confusion matrix at 0.31.

Interpretation. The hold-out AUC (0.96) is higher than the CV estimate (0.90); with 61 patients one case moves accuracy by 1.6 points, so the CV figure is the one to quote. Lowering the threshold to 0.31 removes both missed cases at the cost of six extra false alarms - the intended trade-off for screening. On the external Hungarian site, discrimination largely holds (AUC 0.89) even though `ca`/`thal` are almost entirely imputed, but recall falls to 0.80: the model would need re-calibration before use at another hospital, which the drift monitoring in Section 9 is designed to surface.

5. Experiment tracking (MLflow)

Tracking uses an SQLite backend (`mlflow.db`) with a local artefact store, which also enables the model registry. Experiment `heart-disease-risk` contains **92 runs**:

- **One parent run per model family** - search space, best parameters, CV mean/sd for five metrics, train-CV gap, hold-out metrics, search time; artefacts: `cv_results.csv`, ROC/PR, confusion matrix, calibration and feature-importance plots, and the fitted model with input signature and example.
- **One nested run per grid candidate (88)** - every configuration is visible and comparable (Figure 9 compares all 48 logistic-regression candidates).
- **A champion run** - model card, training config, data provenance (SHA-256), threshold, hold-out and external metrics, and the model registered as `heart-disease-classifier` with alias `@champion`. Models are saved with **skops** and an explicit type allow-list rather than raw pickle, so loading a tampered artefact cannot execute arbitrary code.

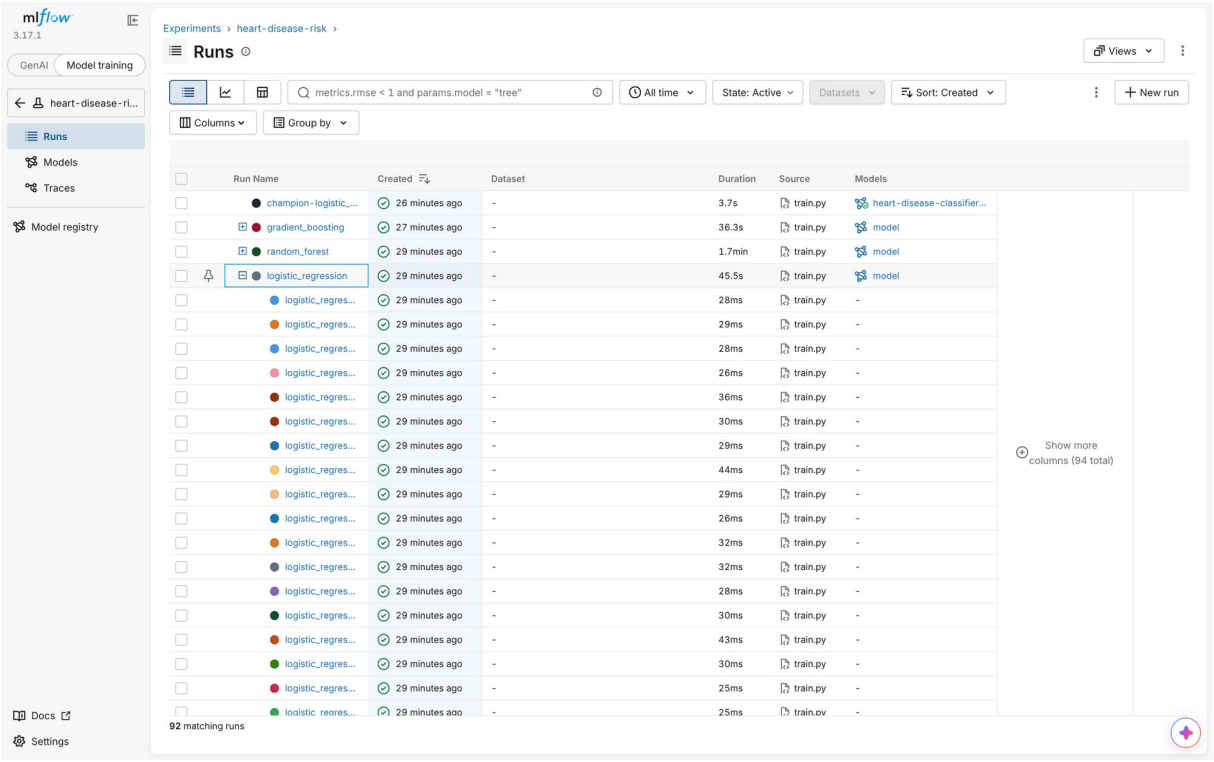


Figure 8. MLflow runs view: family parent runs with the logistic-regression candidates expanded.

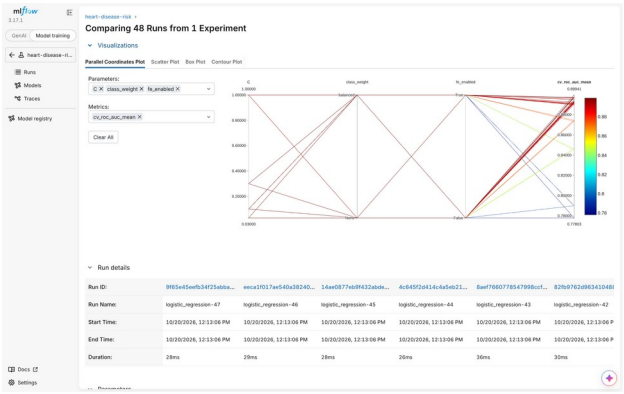


Figure 9. Parallel-coordinates comparison of 48 candidates.

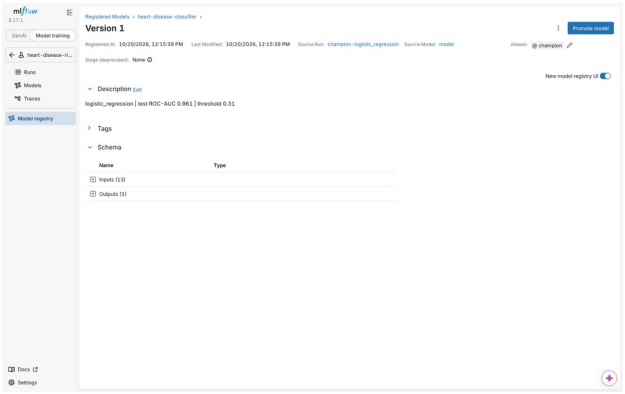


Figure 10. Registered model version with alias @champion.

6. Model packaging and reproducibility

Artefact	Purpose
`models/model.joblib`	Complete fitted pipeline (preprocessing + classifier) loaded by the API
`models/mlflow_model/`	MLflow format: skops-serialised model, input schema, pinned requirements, `predict_proba` pyfunc
`models/model_card.json`	Model type, hyper-parameters, threshold and policy, CV/hold-out/external metrics, training-data SHA-256, training feature profile (used for drift), library versions, intended use
`requirements*.txt`, `environment.yml`	Exact versions (scikit-learn 1.9.1, MLflow 3.17.1, FastAPI 0.150.0, ...); serving image installs only the runtime subset

Artefact	Purpose
`configs/train.yaml`	Every training decision (grids, CV scheme, threshold policy, quality gate), logged with each run

Seeds are fixed for splitting, CV and every estimator; a unit test asserts identical predictions across two fits. Three independent full training runs produced the same CV scores to four decimals.

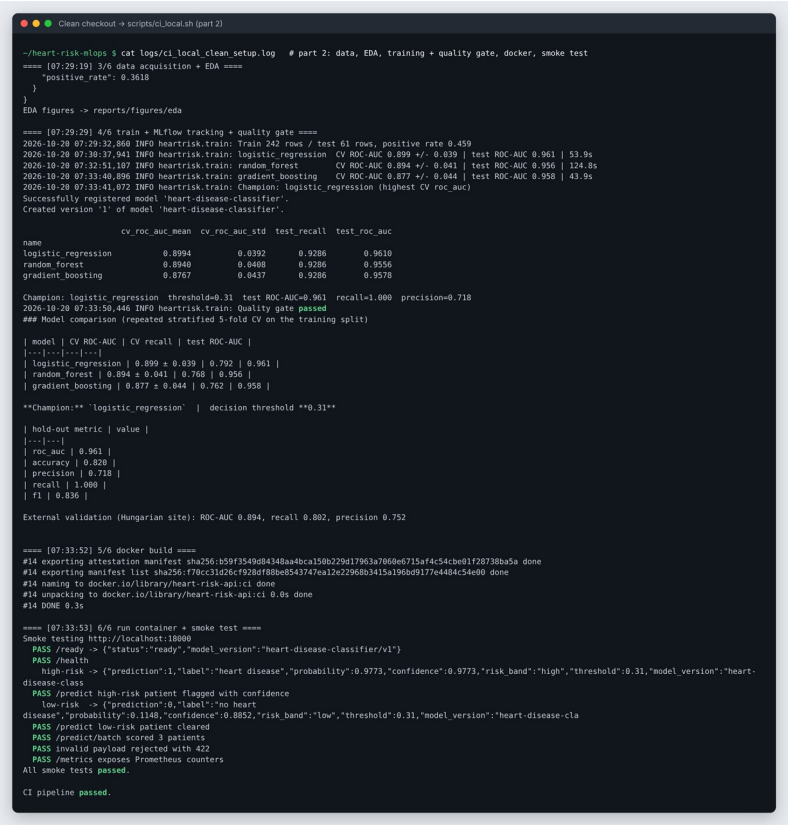


Figure 11. Clean checkout: training, quality gate, Docker build and smoke test (part 2 of the local CI run).

7. CI/CD pipeline and automated testing

Tests (57, 91% line coverage, pytest). Data: parsing of ? markers, binarisation, range nulling, de-duplication, ARFF code mapping, contract violations, checksum-guarded fallback (tampered mirror rejected) - all without network access. Features: engineered values, ablation switch, NaN propagation, no NaN after preprocessing, unseen categories, feature names. Model: every family fits, determinism, serialisation round trip, hand-checked metrics, threshold precision floor, parsimony rule, quality gate, and an end-to-end quick training run into an isolated MLflow store that reloads the registered model. API: contract, confidence semantics, risk ordering, optional fields, 422 validation with field details, batch limits, request-id propagation, Prometheus counters, offline docs, threshold override and the not-ready path. EDA: figures and summary.

GitHub Actions (`.github/workflows/ci-cd.yml`, validated with actionlint):

Job	Steps	Fails when	Artefacts / logging
lint	ruff check, ruff format --check, YAML parse of deploy configs	any lint/format error	inline PR annotations
test	pip install -r requirements.txt; data contract; pytest --cov	any failing test	JUnit XML, coverage XML, log, job summary table
train	download (UCI) -> EDA -> train -> quality gate	gate: hold-out AUC < 0.80 or recall < 0.75	model, MLflow store, reports, logs, metrics summary
docker	buildx build (GHA cache), run, 7-check smoke test; push to GHCR on main	build error, unhealthy container, failed check	container logs, saved image
deploy-kind	kind cluster, kubectl apply -k, rollout status, smoke test via port-forward, helm lint/template/install	rollout timeout or failed check	cluster state, pod logs, rendered Helm

Each script runs with `set -euo pipefail` (or propagates the pytest/training exit code through `tee`), so the pipeline fails on code or test errors with readable logs. The same stages run locally via `scripts/ci_local.sh` (Figure 11).

On GitHub (run #2, commit `27fc7ba`) all five jobs passed in 13 min 49 s and uploaded 7 artefacts; the job summaries publish the test table and the model comparison (Figures 12-13). The first run failed at the smoke test with a clear log (`Permission denied`): committing from Windows had dropped the script's executable bit. The workflow now invokes it through `bash` - the pipeline failing loudly on a real defect, as intended.

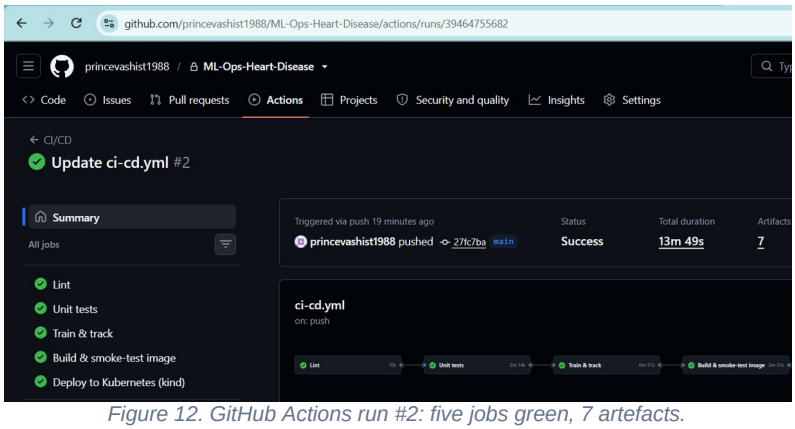


Figure 12. GitHub Actions run #2: five jobs green, 7 artefacts.

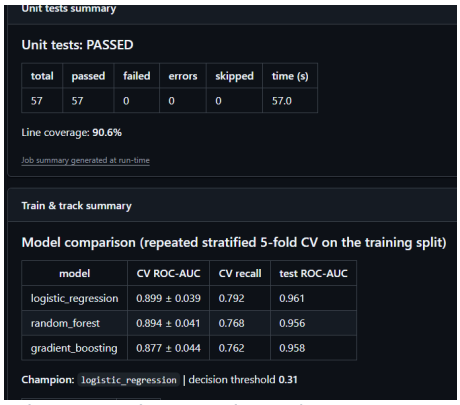


Figure 13. Job summaries: 57/57 tests, 90.6% coverage, model table.

8. Containerisation and production deployment

Docker. A multi-stage build resolves dependencies in a builder stage and copies only the virtual environment, source and model into the runtime stage; the container runs as non-root UID 10001 with a `HEALTHCHECK` on `/ready` and one uvicorn worker (scaling is done with replicas, which keeps Prometheus counters exact per process). `BASE_IMAGE` is a build argument so the image can be built from an internal mirror on air-gapped networks. The API exposes `POST /predict` (JSON in; prediction, label, probability, **confidence** = probability of the predicted class, risk band, threshold, model version, request id out), `/predict/batch`, `/health`, `/ready`, `/model-info`, `/metrics`, `/stats`, `/dashboard` and Swagger `/docs` with vendored assets. Inputs are validated by pydantic with the same physiological bounds as the cleaner; unknown fields are rejected.

```
POST /predict {"age":67, "sex":1, "cp":4, "trestbps":160, "chol":286, "fbs":0, "restecg":2,
               "thalach":108, "exang":1, "oldpeak":1.5, "slope":2, "ca":3, "thal":3}
-> 200 {"prediction":1, "label":"heart disease", "probability":0.9773, "confidence":0.9773,
        "risk_band":"high", "threshold":0.31, "model_version":"heart-disease-classifier/v1", ...}
-> 422 {"detail":[{"field":"age", "message":"Input should be greater than or equal to 18"}]}
```

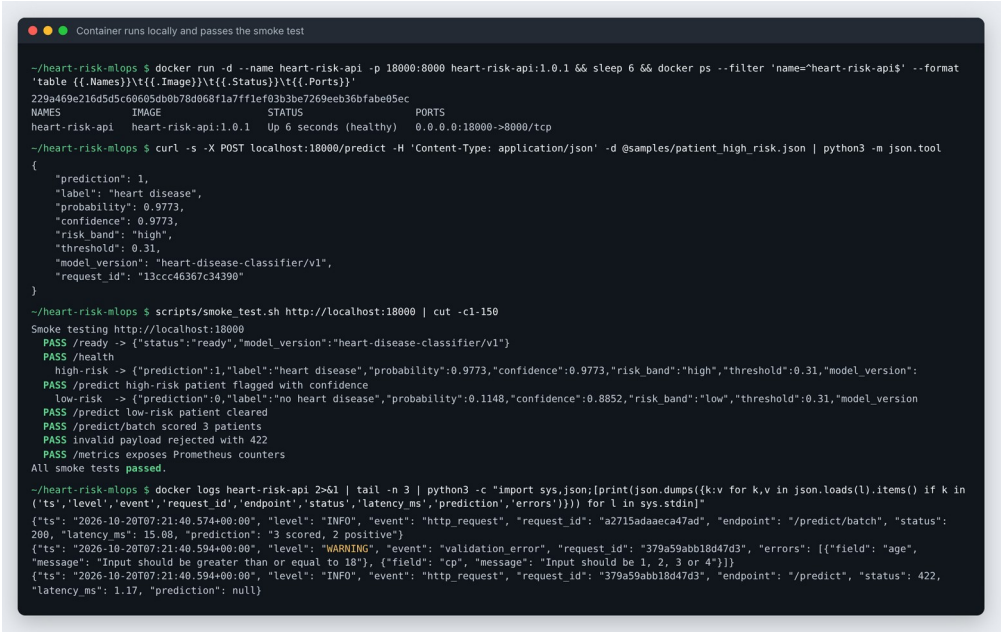


Figure 14. Container built, healthy, answering /predict and passing all smoke checks.

Kubernetes. `deploy/k8s` (kustomize) defines a namespace, ConfigMap, Deployment (2 replicas, `maxUnavailable: 0` rolling updates, startup/readiness/liveness probes, CPU/memory requests and limits, read-

only root filesystem, dropped capabilities, seccomp, pre-stop drain, Prometheus annotations), a LoadBalancer Service, an Ingress (`heart-risk.local`), an HPA (2-5 pods at 70% CPU) and a PodDisruptionBudget. `deploy/helm/heart-risk-api` provides the same deployment as a parameterised chart. The manifests were deployed to a local k3s v1.36 cluster and verified through both the LoadBalancer and the Ingress (Figure 15). A rolling update performed while a probe sent `/predict` every 100 ms completed with **500 successful requests and 0 failures**.

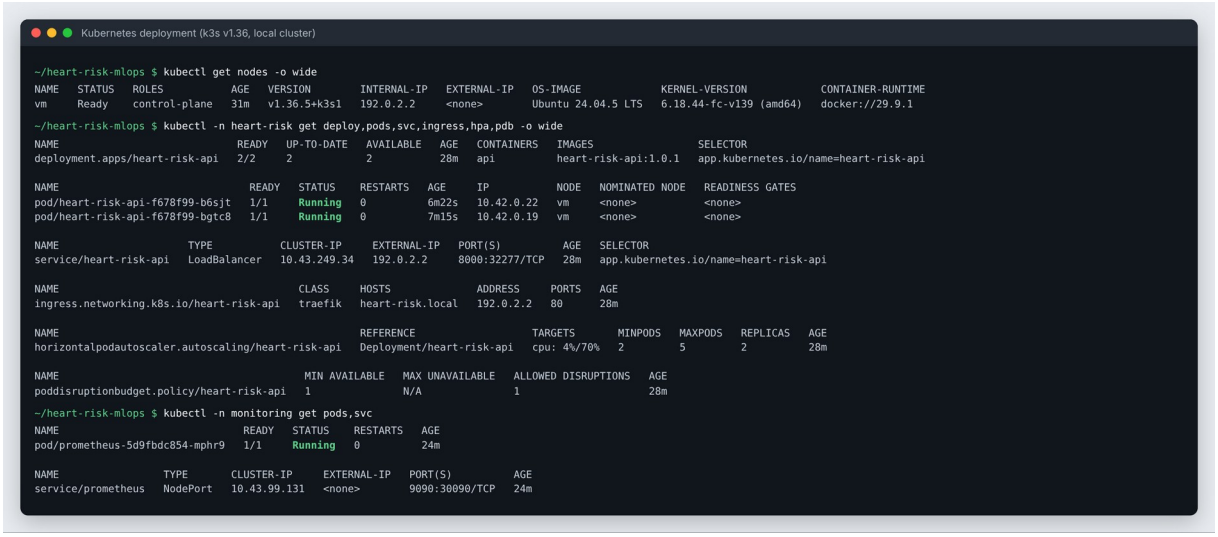


Figure 15. Deployment, pods, LoadBalancer service, ingress, HPA, PDB and the monitoring namespace.

9. Monitoring and logging

- **Request logging**: middleware writes one JSON line per request (timestamp, request id, method, endpoint, status, latency, client, prediction summary) and logs validation failures with field-level detail. Clinical inputs are never logged; an incoming `X-Request-ID` is propagated for tracing.
- **Metrics** (`/metrics`): requests by endpoint and status, latency histograms, in-flight requests, inference time, predictions by outcome and risk band, a probability histogram, validation errors, model info, and live input means exported next to the training means from the model card (a drift signal without storing patient data).
- **Prometheus** discovers every pod via annotations and evaluates five alert rules (API down, model not loaded, 5xx rate > 5%, p95 latency > 250 ms, input drift > 15%). **Grafana** is provisioned with a 16-panel dashboard (`docker compose up` or `kubectl apply -k deploy/monitoring`); the API also serves a dependency-free live dashboard at `/dashboard`.

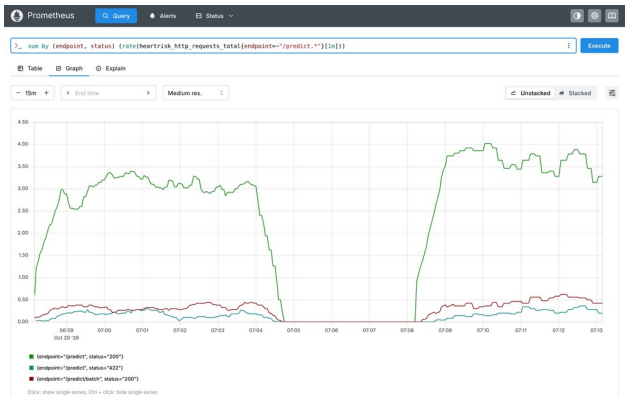


Figure 16. Request rate by endpoint/status (422 = rejected inputs).

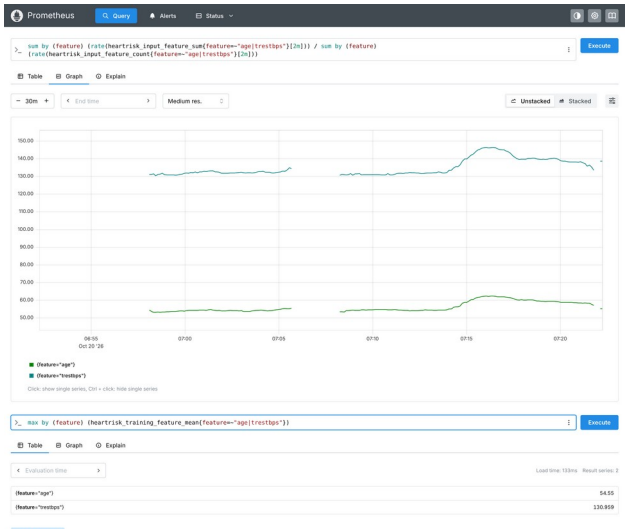


Figure 17. Drift: live mean age/BP rise after drifted traffic (training means 54.6 / 131.0).

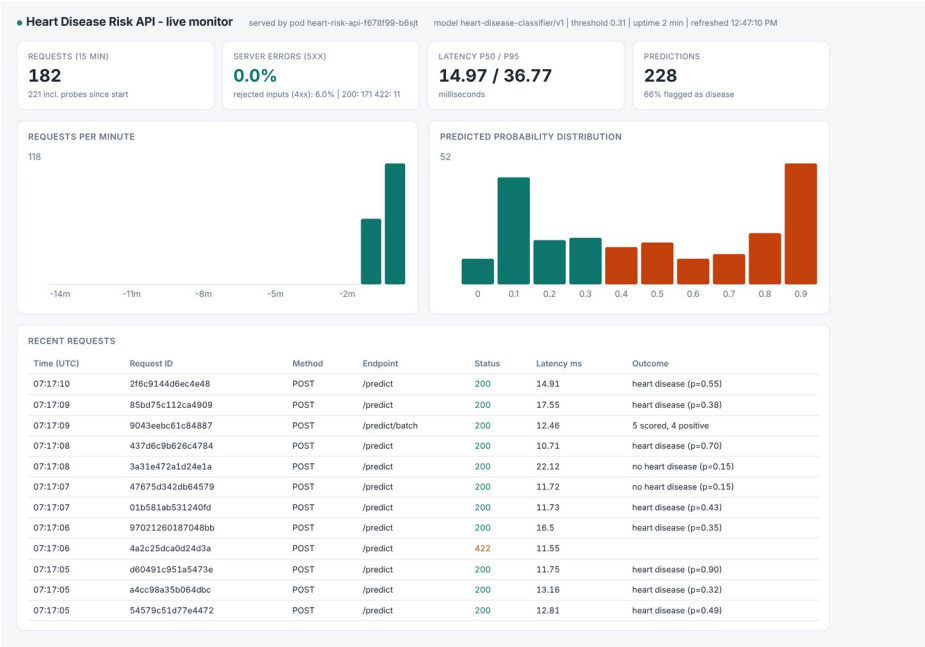


Figure 18. Built-in dashboard during live traffic: 0% server errors, p95 37 ms.

10. Limitations and next steps

- 303 patients from a 1988 cohort: estimates carry wide intervals (CV sd 0.04) and the cohort is 68% male; performance on women and on modern populations is unverified. The model is a teaching prototype, not a medical device.
- Calibration across sites: the external validation shows lower recall; next steps are per-site re-calibration (isotonic/Platt) and conformal prediction intervals.
- Monitoring detects input drift but not performance drift, which needs delayed ground-truth labels; a feedback endpoint and a scheduled re-training job registering a challenger model would close the loop.
- Evidence environment: the local screenshots come from a restricted-network build machine without access to Docker Hub or the UCI host, so the image used a locally bootstrapped Ubuntu 24.04 + Python 3.12 base via `--build-arg BASE_IMAGE` and the Cleveland file came from the checksum-verified mirror (byte-identical). The GitHub Actions run built from the default `python:3.12-slim` base with no overrides and passed, confirming the defaults work unchanged.

Appendix A - Requirement traceability

Task (marks)	Where it is implemented	Evidence
1 Data + EDA (5)	`src/heartrisk/data.py`, `eda.py`, `notebooks/01_eda.ipynb`	Sec. 3, Fig. 2-4, `reports/figures/eda/`
2 Features + models (8)	`features.py`, `models.py`, `train.py`, `configs/train.yaml`	Sec. 4, Fig. 5-7, tables
3 Experiment tracking (5)	MLflow parent/nested/champion runs + registry	Sec. 5, Fig. 8-10, `reports/mlflow_runs.csv`
4 Packaging (7)	`models/`, `requirements*.txt`, `environment.yml`	Sec. 6, Fig. 11
5 CI/CD + tests (8)	`.github/workflows/ci-cd.yml`, `tests/`, `scripts/ci_local.sh`	Sec. 7, Fig. 11-13
6 Containerisation (5)	`Dockerfile`, `src/heartrisk/api/`	Sec. 8, Fig. 14
7 Deployment (7)	`deploy/k8s/`, `deploy/helm/`	Sec. 8, Fig. 15, `screenshots/terminal_09-10`
8 Monitoring + logging (3)	`api/monitoring.py`, `deploy/monitoring/`, `docker-compose.yml`	Sec. 9, Fig. 16-18
9 Documentation (2)	`README.md`, this report, `docs/`	Sec. 1-2, Fig. 1, video `docs/pipeline_walkthrough_video.mp4`